

**Informe técnico de la Versión 3: arquitectura distribuida para el cálculo de velocidades del
SITM-MIO**

María José Betancourt Solano - A00403987

Joshua Sayur Gallego - A00404707

Martin Borrero Herrera - A00403871

Javier Andrés Paz Luna - A00405619

Camilo Andrés Martínez Moreno - A00405205

Universidad Icesi

Ingeniería de Software IV

Proyecto final

Informe técnico de la Versión 3: arquitectura distribuida para el cálculo de velocidades del SITM-MIO

Objetivo

El propósito de la Versión 3 fue llevar el cálculo de velocidades promedio del SITM-MIO a un escenario distribuido. En las versiones anteriores el procesamiento se ejecutaba en una sola máquina; en esta versión, el trabajo se divide entre varios equipos para responder mejor a los atributos de calidad de rendimiento, escalabilidad, correctitud y tolerancia operacional ante datasets más grandes.

La funcionalidad central se conserva: calcular la velocidad promedio de los buses por ruta y por mes a partir de registros GPS. La diferencia principal está en la forma de ejecutar el proceso: el sistema se compone de cliente, broker, master, workers, particionador y visualización, comunicados mediante ZeroC Ice. Ice se utiliza como middleware de comunicación remota orientado a objetos, lo cual permite invocar servicios distribuidos a través de interfaces definidas previamente (ZeroC, 2018).

Método de cálculo

Datos de entrada

La solución usa dos archivos principales de entrada. El primero es datagrams4Pilot.csv, que contiene los registros GPS de los buses. El segundo es lines-241-ActiveGT.csv, que lista las rutas activas que deben considerarse. Además, para validar el comportamiento del sistema en escenarios de diferente tamaño, se realizaron ejecuciones con datagrams-MiniPilot.csv y con el dataset datag. Antes de procesar los datagramas, cada worker carga las rutas activas para descartar registros que no pertenecen al subconjunto solicitado.

Tabla 1
Columnas utilizadas del archivo de datagramas

Índice	Campo	Descripción
[4]	latitude	Latitud GPS guardada como entero; se divide entre 1e7 para obtener grados decimales.
[5]	longitude	Longitud GPS guardada como entero; se divide entre 1e7 para obtener grados decimales.
[7]	lineId	Identificador de la ruta en la que se encuentra el bus.
[10]	datagramDate	Fecha y hora del datagrama con formato yyyy-MM-dd HH:mm:ss.

[11]	busId	Identificador del bus.
------	-------	------------------------

Nota. Estas columnas permiten identificar la ruta, el bus, la posición geográfica y el momento en que se produjo cada registro.

Descripción del algoritmo

El primer paso consiste en dividir el dataset en particiones. Para no perder correctitud, la partición se realiza considerando la clave busId|lineId y el corte por día. Esta decisión es crítica: si los registros de un mismo bus, una misma ruta y un mismo día quedaran separados en workers distintos, se rompería la comparación entre posiciones consecutivas y el cálculo de velocidad podría quedar incompleto o sesgado. Además, el filtrado por día facilita acumular velocidades diarias antes de consolidarlas por mes.

Cada worker recibe una partición y la recorre de forma secuencial. En lugar de guardar todas las posiciones en memoria, conserva únicamente la última posición conocida para cada combinación busId|lineId. Cuando llega una nueva posición del mismo grupo, calcula el tiempo transcurrido, la distancia recorrida y la velocidad. Este enfoque reduce el consumo de memoria y es más adecuado para un dataset piloto de mayor tamaño.

El cálculo se hace primero por día y luego por mes. Las velocidades válidas se acumulan por ruta y día; después, esos acumulados diarios se consolidan para obtener el promedio mensual por ruta. Se descartan los pares de puntos con tiempo menor o igual a cero, distancia menor o igual a cero o velocidad superior a 80 km/h. El descarte de velocidades superiores al umbral evita que errores de GPS, saltos de posición o registros inconsistentes distorsionen el promedio final.

Fórmula Haversine

La distancia entre dos coordenadas GPS se calcula mediante la fórmula Haversine, usada comúnmente para estimar distancias de círculo máximo entre dos puntos sobre una esfera (Sinnott, 1984):

$$a = \sin^2(\Delta\text{lat}/2) + \cos(\text{lat}1) \times \cos(\text{lat}2) \times \sin^2(\Delta\text{lon}/2)$$

$$\text{distancia} = 6371 \times 2 \times \text{atan2}(\sqrt{a}, \sqrt{1-a}) \text{ [km]}$$

En esta fórmula, las latitudes y longitudes se expresan en radianes. El valor 6371 corresponde al radio aproximado de la Tierra en kilómetros. La fórmula es suficiente para el objetivo del

proyecto, aunque no modela con precisión el elipsoide terrestre; por tanto, su resultado debe entenderse como una aproximación práctica, no como una medición geodésica exacta.

Optimizaciones aplicadas

La Versión 3 no solo distribuye el procesamiento, sino que también ajusta la forma en que cada worker maneja la información. Esto fue necesario porque el dataset piloto es más grande que el utilizado para las pruebas iniciales y cargar todas las posiciones en memoria podía producir errores de memoria.

Tabla 2
Optimizaciones implementadas en la Versión 3

Optimización	Descripción
Particionamiento por bus, ruta y día	Mantiene juntas las posiciones que deben compararse para calcular velocidades y respeta el corte diario antes de la consolidación mensual.
Procesamiento distribuido	Reparte particiones entre varios workers para reducir el trabajo individual por equipo.
Procesamiento por streaming	Evita guardar listas completas de datagramas; cada worker conserva solo la última posición por grupo y acumulados parciales.
Acumulación diaria y consolidación mensual	Calcula primero por ruta y día, y luego consolida por ruta y mes.
Visualización filtrada	Permite observar eventos y recorridos sin saturar el mapa.

Nota. Las optimizaciones se enfocan en rendimiento, uso de memoria y escalabilidad sin cambiar el resultado funcional esperado.

Resultados

La ejecución distribuida se validó con 10 workers activos en tres escenarios: el dataset completo datagrams4Pilot.csv, el dataset datagrams-MiniPilot.csv y el dataset datag. En todos los casos, el cliente envía la tarea al broker, el broker la enruta al master y el master reparte 10 particiones entre los workers registrados. Cada worker procesa su partición y devuelve estadísticas parciales, que luego son consolidadas por el master.

Ejecución con 10 workers

En la evidencia actualizada con datagrams4Pilot.csv, la tarea v3-flex-demo finalizó correctamente con Success: true. La consola reportó 10 workers activos, 10/10 particiones completadas, 1356 registros generados, salida output/resultados-v3.csv y tiempo total de 70 999 ms.

Tabla 3
Evidencias de la ejecución distribuida con 10 workers

Elemento	Evidencia observada
Workers	10 workers activos y procesamiento de 10/10 particiones.
Cliente	Finalización de la tarea con Success: true.
Salida	Archivo output/resultados-v3.csv reportado y escrito localmente.
Registros	1356 registros generados en la salida final.
Tiempo reportado	70 999 ms en la ejecución con datagrams4Pilot.csv.

Nota. Los valores corresponden a la ejecución evidenciada en la Figura 1.

Figura 1

Ejecución distribuida de la Versión 3 con 10 workers activos.

```
swarch@104m31:~/Documents/JCMMJ$ bash v3-distributed/start-node.sh client
Jun 12, 2026 12:36:10 PM co.icesi.project.v3.client.ClientApp main
INFO: Submitting task v3-flex-demo through Broker
Task: v3-flex-demo
Success: true
Message: Distributed calculation completed with 10 active workers and 10/10 completed partitions
Output: output/resultados-v3.csv
Records: 1356
ElapsedMs: 70999
Local output written: output/resultados-v3.csv
```

Nota. La captura valida que el cliente recibe una respuesta exitosa de la tarea v3-flex-demo: Success: true, 10 workers activos, 10/10 particiones completadas, 1356 registros, tiempo de 70 999 ms y salida output/resultados-v3.csv.

Versión 3 con MiniPilot y 10 workers

También se ejecutó la Versión 3 con la misma arquitectura distribuida y 10 workers sobre datagrams-MiniPilot.csv. Esta corrida es importante porque produce 97 registros, el mismo volumen de salida reportado para V1 y V2. Por eso sirve como referencia más cercana para comparar el comportamiento de V3 frente a las versiones monolíticas, aunque sigue incluyendo costos propios de comunicación y coordinación distribuida.

Tabla 4

Evidencias de la ejecución distribuida con 10 workers usando MiniPilot

Elemento	Evidencia observada
Workers	10 workers activos y procesamiento de 10/10 particiones.
Cliente	Finalización de la tarea con Success: true.
Salida	Archivo output/resultados-v3.csv reportado por el flujo distribuido.
Registros	97 registros generados en la salida final.
Tiempo reportado	1 590 ms en la ejecución con datagrams-MiniPilot.csv.

Nota. Esta evidencia usa el mismo número de registros de salida reportado en las versiones monolíticas, pero sigue sin ser un benchmark definitivo porque no controla todos los factores de infraestructura, caché, red y repetición experimental.

Figura 2

Ejecución distribuida de la Versión 3 con MiniPilot y 10 workers activos.

```
swarch@104m31:~/Documents/JCMMJ$ bash v3-distributed/start-node.sh client
Jun 12, 2026 12:46:14 PM co.icesi.project.v3.client.ClientApp main
INFO: Submitting task v3-flex-demo through Broker
Task: v3-flex-demo
Success: true
Message: Distributed calculation completed with 10 active workers and 10/10 completed partitions
Output: output/resultados-v3.csv
Records: 97
ElapsedMs: 1590
Local output written: output/resultados-v3.csv
```

Nota. La captura evidencia la ejecución de la Versión 3 con MiniPilot: Success: true, 10 workers activos, 10/10 particiones completadas, 97 registros, tiempo de 1 590 ms y salida local output/resultados-v3.csv.

Versión 3 con datag y 10 workers

Finalmente, se ejecutó la misma arquitectura distribuida con 10 workers sobre el dataset datag. Esta corrida generó 1 registro y terminó con Success: true, 10 workers activos, 10/10 particiones completadas, salida output/resultados-v3.csv y tiempo de 788 ms. El resultado es útil como evidencia funcional de extremo a extremo, pero no como prueba fuerte de rendimiento, porque con tan pocos registros domina el costo fijo de coordinación entre cliente, broker, master y workers.

Tabla 5

Evidencias de la ejecución distribuida con 10 workers usando datag

Elemento	Evidencia observada
Workers	10 workers activos y procesamiento de 10/10 particiones.
Cliente	Finalización de la tarea con Success: true.
Salida	Archivo output/resultados-v3.csv reportado por el flujo distribuido.
Registros	1 registro generado en la salida final.
Tiempo reportado	788 ms en la ejecución con datag.

Nota. Esta ejecución confirma que la arquitectura puede completar el flujo distribuido incluso con un dataset muy pequeño; sin embargo, por producir un solo registro, su valor es principalmente funcional y no comparativo.

Figura 3

Ejecución distribuida de la Versión 3 con datag y 10 workers activos.

```
swarch@104m31:~/Documents/JCMMJ$ bash v3-distributed/start-node.sh client
Jun 12, 2026 12:53:18 PM co.icesi.project.v3.client.ClientApp main
INFO: Submitting task v3-flex-demo through Broker
Task: v3-flex-demo
Success: true
Message: Distributed calculation completed with 10 active workers and 10/10 completed partitions
Output: output/resultados-v3.csv
Records: 1
ElapsedMs: 788
Local output written: output/resultados-v3.csv
```

Nota. La captura evidencia la ejecución de la Versión 3 con datag: Success: true, 10 workers activos, 10/10 particiones completadas, 1 registro, tiempo de 788 ms y salida local output/resultados-v3.csv.

Comparación con las versiones anteriores

La comparación debe leerse con cuidado. MiniPilot es el punto más comparable entre V1, V2 y V3 porque las tres ejecuciones reportan 97 registros. Además, en este informe se incluyen dos

evidencias adicionales de la Versión 3: una con MiniPilot (Figura 2) y otra con datag (Figura 3). La ejecución con datagrams4Pilot.csv muestra el escenario de mayor carga, mientras que datag sirve sobre todo para validar el flujo distribuido con un dataset mínimo.

Tabla 6
Comparación técnica entre V1, V2 y V3

Versión	Arquitectura	Dataset / salida	Tiempo reportado	Lectura técnica
V1	Monolítica MVC secuencial	MiniPilot / 97 registros	13 045 ms	Base funcional simple. Fácil de validar, pero limitada en rendimiento y escalabilidad.
V2	MVC con concurrencia local	MiniPilot / 97 registros	11 128 ms	Mejora el cálculo local con hilos. No distribuye el procesamiento entre equipos.
V3 con MiniPilot (10 workers)	Distribuida: Broker y Master-Worker	datagrams-MiniPilot.csv / 97 registros	1 590 ms	Es la comparación más cercana con V1 y V2. Reduce el tiempo reportado, aunque mantiene el overhead propio de una arquitectura distribuida.
V3 con datag (10 workers)	Distribuida: Broker y Master-Worker	datag / 1 registro	788 ms	Nuestro flujo distribuido logra un buen tiempo bajo el análisis de una ruta
V3 con datagrams4Pilot (10 workers)	Distribuida: Broker y Master-Worker	datagrams4Pilot.csv / 1356 registros	70 999 ms	Representa el escenario de mayor carga. Su principal aporte es mostrar escalabilidad y mejor control de memoria por worker.

Frente a V1, la Versión 3 cambia el problema de un procesamiento secuencial centralizado a un flujo distribuido. Frente a V2, ya no se trata solo de paralelismo dentro de una máquina, sino de coordinación entre nodos. En MiniPilot, V3 reportó 1 590 ms con 97 registros, lo que sugiere una mejora relevante frente a V1 y V2; en datag, V3 reportó 788 ms con 1 registro.

Arquitectura del sistema

La solución se organiza como una arquitectura distribuida por componentes. Cada módulo tiene una responsabilidad específica y se comunica con los demás mediante interfaces remotas definidas en el módulo slice.

Tabla 7
Responsabilidades de los componentes de la Versión 3

Componente	Responsabilidad
------------	-----------------

Client	Construye la tarea de cálculo y la envía al broker.
Broker	Recibe la solicitud del cliente y la reenvía al master.
Master	Registra workers, asigna particiones, recibe resultados parciales y genera el CSV final.
Worker	Procesa una partición y calcula velocidades parciales.
Partitioner	Divide el dataset en particiones usando la clave busId lineId y el corte por día.
Visualization	Recibe eventos y muestra el comportamiento del sistema y las posiciones de buses.
Slice	Define los contratos remotos usados por los componentes.

Nota. Los componentes separan coordinación, procesamiento y observabilidad.

Los patrones arquitectónicos más relevantes son Master-Worker, particionamiento de datos, Map-Reduce simplificado y comunicación orientada a eventos. Estos patrones son reconocibles en la estructura del código, en los nombres de los módulos y en el flujo de ejecución: client -> master -> workers -> master -> resultado.

Despliegue

El despliegue se realizó sobre varios computadores del laboratorio. El equipo de visualización y cliente ejecuta la interfaz y lanza la tarea. El equipo coordinador ejecuta master, broker y particionador. Los demás equipos ejecutan workers. La configuración se centraliza en el archivo v3-distributed/deploy.env.

En la configuración actual se usan los puertos 11000 para broker, 11001 para master y 11003 para visualización. Los workers usan puertos derivados de WORKER_BASE_PORT=11010. Esta configuración evita conflictos con procesos anteriores y facilita repetir el despliegue.

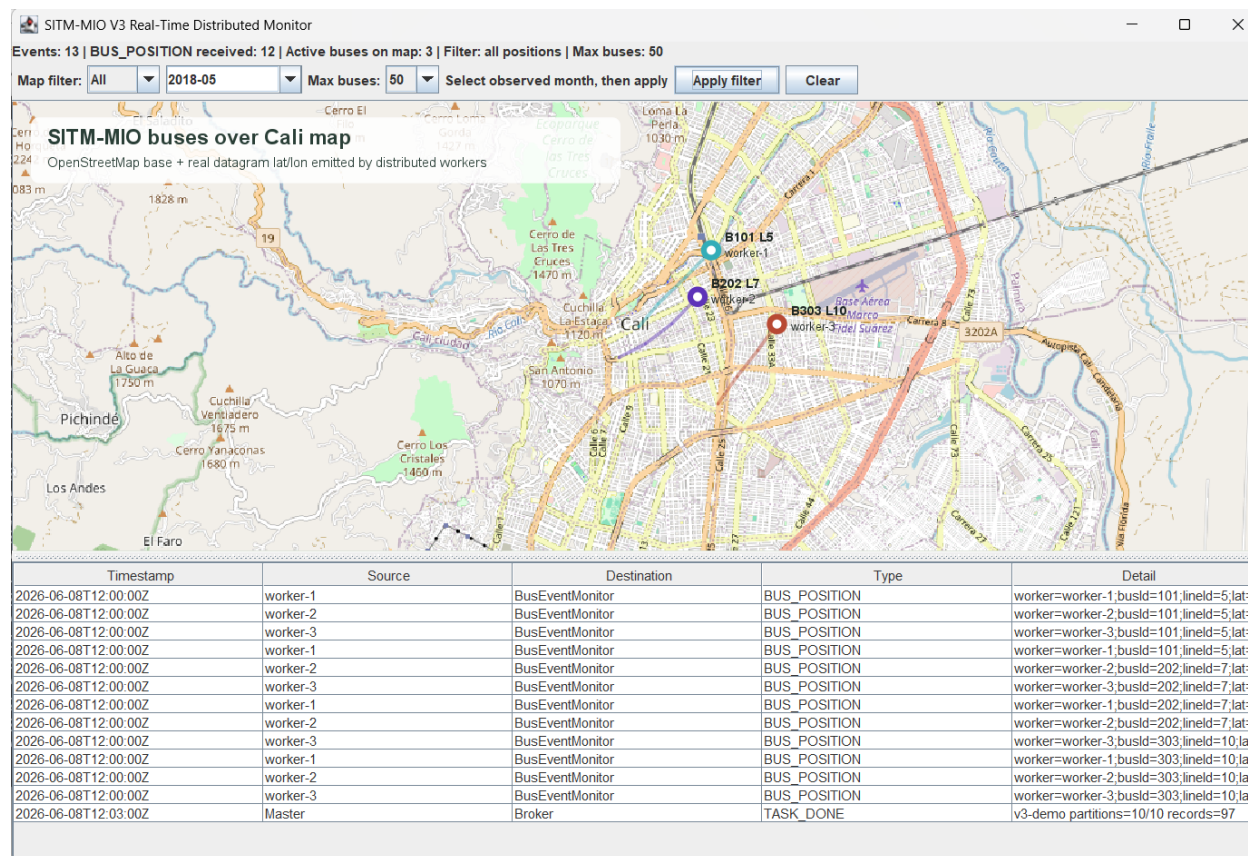
Visualización

Los workers publican eventos BUS_POSITION con el identificador del bus, la ruta, las coordenadas, la fecha del datagrama y el worker que procesó el evento.

Para evitar que el mapa se sature, la interfaz permite filtrar por mes y limitar la cantidad máxima de buses renderizados. Esta decisión no altera el cálculo final, porque el procesamiento completo sigue ejecutándose en los workers; únicamente se controla cuánta información se dibuja en pantalla durante la demostración. Cuando se usan mapas de OpenStreetMap, la atribución debe ser visible para los usuarios del mapa o del material producido (OpenStreetMap Foundation, 2022).

Figura 4

Visualización de buses sobre el mapa de Cali durante la ejecución distribuida de la Versión 3.



Nota. La figura muestra eventos BUS_POSITION recibidos por la visualización y renderizados sobre un mapa de Cali. La interfaz permite filtrar por mes y limitar el número de buses mostrados, lo cual mejora la observabilidad sin modificar el cálculo final.

Validación

La implementación fue validada mediante compilación Gradle y ejecución del flujo distribuido completo. La validación incluyó el registro de workers, el envío de la tarea desde el cliente, la consolidación de resultados en el master y la recepción de eventos en la visualización.

Comandos de validación ejecutados:

```
.\gradlew.bat :v3-distributed:visualization:installDist :v3-
distributed:client:installDist :v3-distributed:master:installDist :v3-
distributed:broker:installDist :v3-distributed:worker:installDist :v3-
distributed:partitioner:installDist
```

```
bash v3-distributed/start-node.sh visualization
bash v3-distributed/start-node.sh worker N
bash v3-distributed/start-node.sh coordination
```

```
bash v3-distributed/start-node.sh client
```

Limitaciones metodológicas

La evidencia disponible permite validar el flujo funcional y distribuido, pero no reemplaza una evaluación completa de rendimiento. La corrida con MiniPilot permite una comparación más cercana con V1 y V2 por el número de registros de salida. La corrida con datag confirma que el flujo distribuido también funciona con un dataset pequeño, aunque su tiempo está dominado por costos fijos de coordinación. Para afirmar mejoras cuantitativas sólidas sería necesario ejecutar V1, V2 y V3 con el mismo dataset, la misma máquina o infraestructura controlada, varias repeticiones por versión y métricas separadas de I/O, cálculo, red y consolidación.

También debe considerarse que el particionamiento por busId|lineId y día protege la correctitud del cálculo, pero puede producir particiones desbalanceadas si algunas rutas, buses o días concentran muchos más datagramas que otros. En ese caso, el tiempo total de la ejecución distribuida queda limitado por el worker más lento.

Aun así, puede sostenerse que V3 responde mejor al problema del dataset completo datagrams4Pilot.csv, porque la motivación de la versión distribuida fue superar el cuello de botella que aparece al procesar el datagrama completo en versiones monolíticas. Esta afirmación es arquitectónicamente razonable, pero debería reforzarse con mediciones controladas antes de presentarla como conclusión experimental fuerte.

Conclusiones

La Versión 3 permite procesar el dataset piloto con una arquitectura más escalable que las versiones anteriores. La división del trabajo en particiones reduce la carga individual de cada equipo y permite experimentar con distintas cantidades de workers.

El particionamiento por busId|lineId y día fue una decisión clave para mantener la correctitud, porque conserva juntas las posiciones que deben compararse y respeta el cálculo diario antes de la consolidación mensual. Además, el procesamiento por streaming redujo el consumo de memoria y permitió que los workers procesaran particiones grandes de forma más estable.

Las ejecuciones con 10 workers confirman que el flujo distribuido puede completar la tarea con todas las particiones asignadas. En datagrams4Pilot.csv se obtuvieron 1356 registros en 70 999 ms; en datagrams-MiniPilot.csv se obtuvieron 97 registros en 1 590 ms; y en datag se obtuvo 1

registro en 788 ms. Esta diferencia muestra que el tiempo total depende del tamaño del dataset y también del overhead fijo de la arquitectura distribuida.

La visualización complementa la solución porque permite observar la ejecución distribuida y los recorridos de buses sin intervenir en el resultado final. En conjunto, la Versión 3 responde mejor a los drivers de performance y escalabilidad planteados para el proyecto, aunque su desempeño debe medirse de forma controlada antes de presentar conclusiones cuantitativas fuertes frente a V1 y V2.

Referencias

OpenStreetMap Foundation. (2022). Licence/Attribution guidelines. OpenStreetMap Foundation.
https://osmfoundation.org/wiki/Licence/Attribution_Guidelines

Sinnott, R. W. (1984). Virtues of the Haversine. *Sky and Telescope*, 68(2), 159.

ZeroC, Inc. (2018). Ice 3.7.1 documentation: Distributed programming with Ice.
<https://download.zeroc.com/ice/3.7/Ice-3.7.1.pdf>